Usage of _

- As mentioned in the 1st lecture, _ can be used in the name of a variable, while there are some special usages

1. Represent the last expression in the interpreter

```
In [1]: 1+1
Out[1]: 2

In [2]: _
Out[2]: 2
```

2. Represent the values that we don't care

```
In [7]: _
Out[7]: [2, 3, 4, 5]

In [8]: a,*_,b=[1,2,3,4,5,6]

In [9]: a
Out[9]: 1

In [10]: b
Out[10]: 6

In [11]: _
Out[11]: [2, 3, 4, 5]
```

3. Visual separator for digit grouping purposes

```
In [12]: amount = 1_000_000.1

In [13]: amount
Out[13]: 1000000.1
```

4. Single trailing underscore var_

avoid conflicts with Python keywords.

```
In [14]: import keyword
    ...: print(keyword.kwlist)
['False', 'None', 'True', '__peg_parser__', 'and',
'as', 'assert', 'async', 'await', 'break', 'class',
'continue', 'def', 'del', 'elif', 'else', 'except',
'finally', 'for', 'from', 'global', 'if', 'import',
'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass',
'raise', 'return', 'try', 'while', 'with', 'yield']
```

5. Single leading underscore _var

_ in front of a variable or method name is **a weak internal use indicator.** It warns the developer that this variable, method, or function is not supposed to be imported and used publicly. It is commonly used to present private variables in a class.

- According to PEP8, single leading underscore **_var is intended for internal use**. Hence, ***from M import** ** doesn't import objects whose names start with an underscore.

However, in general, the single leading underscore is only a naming convention to indicate the variable or function is for internal use. Programmers are still able to import the name if they really want.

- You can still import _var explicitly like **from M import _var** or import M and use **M._var** to access it.

6. Double Leading Underscore __var

- Python interpreter will do name mangling to identifiers with leading underscores. Name mangling is the process to overwrite such identifiers in a class to **avoid conflicts of names between the current class and its subclasses**.

1. Attribute ***__var*** defined in the class cannot be directly accessed outside class and it is replaced with ***_ClassName__var***.
2. Attribute ***__var*** canot be overwritten in the subclasses.

- Unlike single trailing underscore, there is no special meaning for double trailing underscores. You can probably think of ***var__*** as an extension of ***var_***.
- However, double leading and trailing underscore ***__var__*** is completely different and it is a very important pattern in Python.

7. Double Leading/Trailing Underscore __var__

- Python does not apply name mangling to such attributes ***__var__***, but names with double leading and trailing underscores are reserved for special use in Python. They are called Magic Names. You can check Python documentation to get a list of magic names. Names like ***__init__***, ***__call__***, ***__slots__***, ***__name__***, ***__main__*** are all magic names.
- These magic attributes and magic methods are allowed to be overwritten, but you need to know what you are doing. For example, ***__str__*** is a method that returns the string representation of the object. It is called in ***print()*** or ***str()***
- You are allowed to have a customized name with double leading and trailing underscore like ***__day__***. Python will just take it as a regular attribute name and will not apply name mangling on it. However, it's not a good practice to have a name like this without a strong reason.

Summary of _

- Single standalone underscore _ & Single leading underscore _var & Single trailing underscore var_.

Patterns with a single underscore are basically naming conventions. Python interpreter doesn't prevent them from being imported and used outside the module/class.

- Double leading underscores __var & Double leading and trailing underscores __var__.

Patterns with double underscores are more strict. They are either not allowed to be overwritten or need a good reason to do so. Please notice that there is no special meaning for double trailing underscores var__.

Data structures

- In codes, there might be different types of variables, and sometimes it is more convenient to divide them into different groups. In python, there are many ways to group multiple values together in a collection.
- The common ways are **list**, **tuple**, **set**, **range** and **dictionary**.
- Here are some examples.

```
# a list of strings
month_names = ['Jan', 'Feb', 'Mar', 'Apr']

# a list of integers
numbers = [1, 7, 4, 10, 162]

# an empty list
my_list = []

# a list of variables we defined somewhere else
things = [var1, var2, var3]
```

List

- **List**, is a list of quantities, one after another. The quantities in a list are called its elements, and they do not have to be the same type.

- For a list, r =[1, 7, 10, 15]. The individual elements are denoted r[0], r[1], r[2], and so forth. Crucially, the numbers start from ***zero***, not one.

- We can have high dimensional list with list as the elements, however, there might be some unexpected errors. BE CAREFUL!

- A special functions for lists is the function ***map***, which allows you apply ordinary functions to all the elements of a list at once. It is can be combined with ***lambda*** function.

lambda function

- A lambda function is a small anonymous function. It can take any number of arguments, but can only have one expression.
  lambda *arguments : expression*
  The expression is executed and the result is returned.
- The power of lambda is better shown when you use them as an anonymous function inside another function. For example, you have a function definition that takes one argument, and that argument will be multiplied with an unknown number:

```
def myfunc(n):
  return lambda a : a * n

mydoubler = myfunc(2)
mytripler = myfunc(3)

print(mydoubler(11))
print(mytripler(11))
```

Claim or Initialize a list

- By hand

A=[[None, None], [None, None], None]

- Use other lists

A=B; A=copy(B); A=B.copy()

A=B+C

This could give you serious mistakes. BE CAREFUL!

- Use multiplication operator (mean copy)

A=[None]*2; A=[[None]*2]*3

This could give you serious mistakes. BE CAREFUL!

- Use **for** loops

A =[None for x in range(n)]

A =[[None for x in range(n)] for y in range(m)]

A =[[[None for x in range(n)] for y in range(m)] for z in range(p)]

Examples: 1D list

- T=[0,1,2,3,4]

B=T
C=T.copy()
D=T[:]

# 1. If we set T[0:5]=[1,1,1,1,1]
B=?
C=?
D=?

# 2. if we set T=[1,1,1,1,1]??
B=?
C=?
D=?

# 3. if we set B[0:5]=[1,1,1,1,1]
T=?
C=?
D=?

# 4. if we set C[0:5]=[1,1,1,1,1]
T=?
B=?
D=?

# 5. if we set D[0:5]=[1,1,1,1,1]
T=?
B=?
C=?

Why??

Examples: 2D list

- T2=[0,1,2,3,4]

B2=[None,None]; B2[0]=T2; B2[1]=T2.copy(); B2[1]=T2;
B2.append(T2)
C2=B2.copy(); D2=B2[:]; E2=B2

#1. If we set T2[0:5]=[1,1,1,1,1]; B2[0][1]='*'; B2[1][0]='#'
T2=?; B2=?; C2=?; D2=?; E2=?

#2. If we set T2=[1,1,1,1,1]; B2[0][1]='*'; B2[1][0]='#'
T2=?; B2=?; C2=?; D2=?; E2=?

#3. If we set T2[0:5]=[1,1,1,1,1]; C2[0][1]='*'; C2[1][0]='#'
T2=?; B2=?; C2=?; D2=?; E2=?

Why??

Examples: use multiplication

#1. A=[0, 0,0]; A[0]=1;
A=?

#2. A=[0]*3; A[0]=1;
A=?

#3. A=[[0,0,0], [0,0,0]]; A[0][0]=1;
A=?

#4. A=[[0]*3]*2; A[0][0]=1;
A=?

#5. A=[[0]*3]*2; A[0][1:2]=[1,2];
A=?

#6. A=[[0]*3]*2; A[0]=1;
A=?

Why??

A good tool/function id()

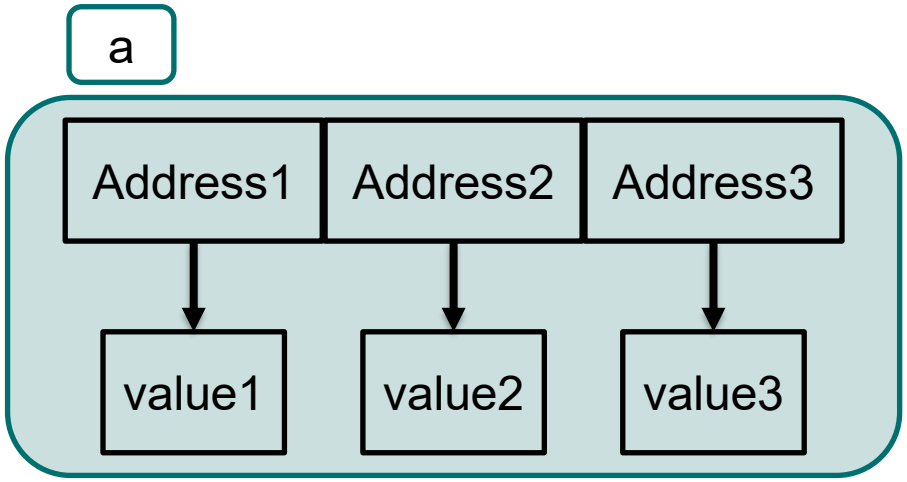
- id(object): return the address of the object in the memory. The object could be anything. Is it a fixed number?

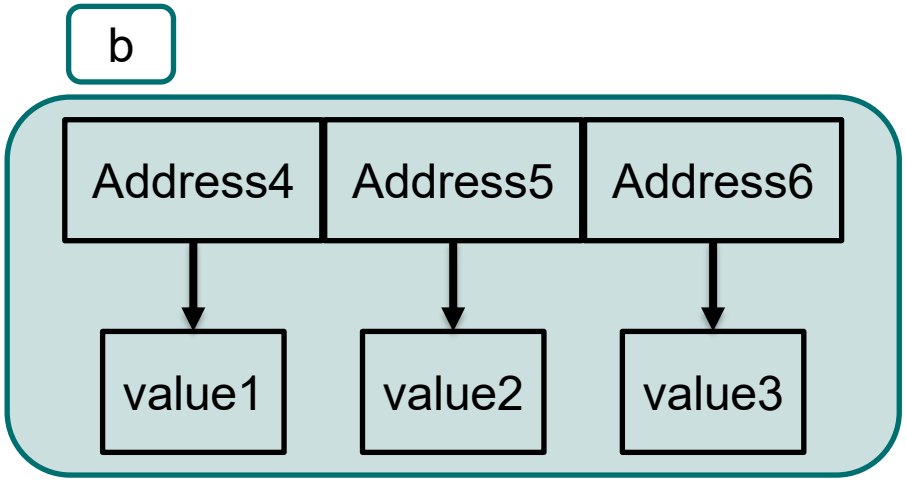
a=1
b=1
Q1: id(a)=?
Q2: id(b)=?
Q3: id(1)=?

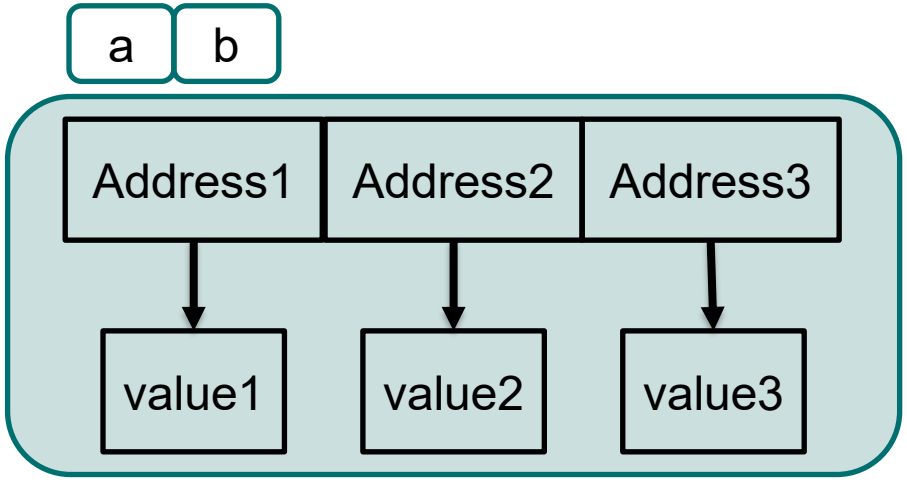
a=[1]
b=[1]
Q4: id(a)=?
Q5: id(b)=?
Q6: id(a[0])
Q7: id(b[0])

Shallow copy and deep copy

- All the confusions come from the differences between shallow copy and deep copy. They might have different names like value copy(值复制) and address copy(位复制).







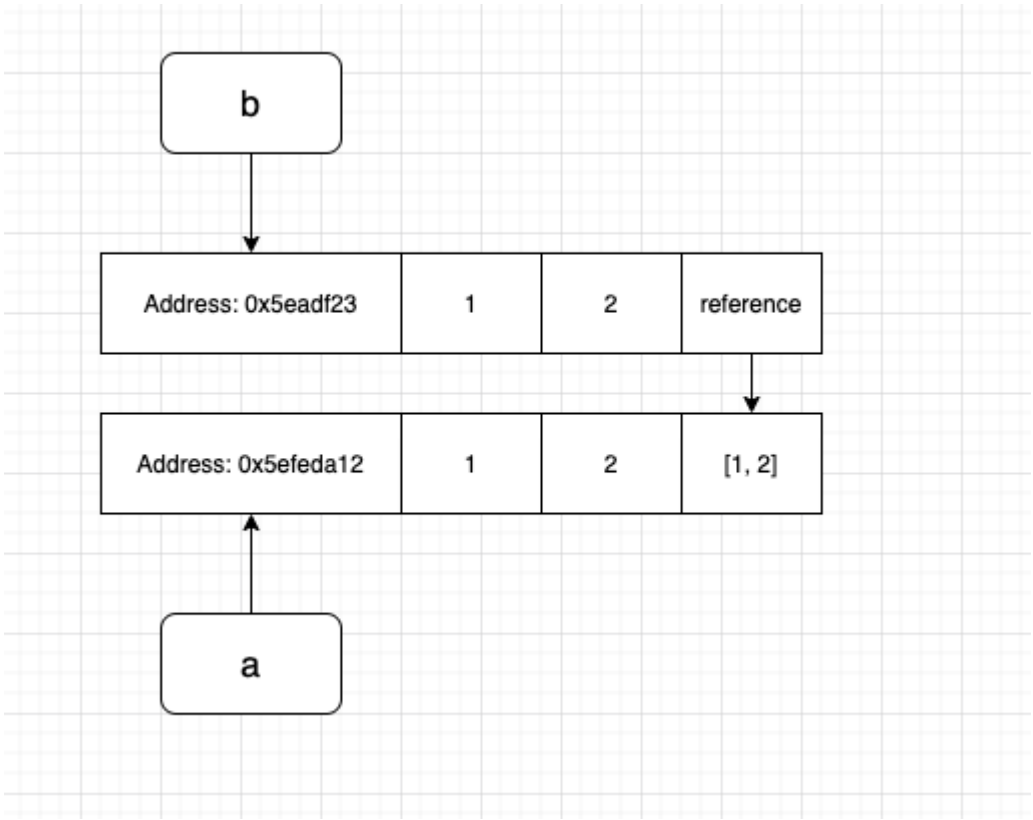
- copy.copy() and copy.deepcopy()
- Be careful and do you own test before you use it.

Shallow copy

- The problem with shallow copy is it does not copy a nested object, in this case, it is [1, 2] which is nested in the original list.

```
import copy
a = [1, 2, [1, 2]]
b = copy.copy(a)
b[2][0] = 11

print("list a:", a)
print("list b:", b)
```



Check the size of memory used

```
import sys
import matplotlib.pyplot as plt

a=1
print(a.__sizeof__())
print(sys.getsizeof(1))

a=[1]
print(a.__sizeof__())
print(sys.getsizeof(a))

a=['sjdfosajfsaodfsaojdfsjaodfjasdfjso']
print(sys.getsizeof(a))
# sys.getsizeof only take account of the list itself, not items it contains.

list_size=[]
size_l=[]
mem_l=[]
size_l.append(len(list_size))
mem_l.append(sys.getsizeof(list_size))
print(sys.getsizeof(list_size))
for n in range(100):
    list_size.append(n)
    # print(list_size)
    # print(id(list_size),sys.getsizeof(list_size))
    size_l.append(len(list_size))
    mem_l.append(sys.getsizeof(list_size))

plt.plot(size_l,mem_l,'*-')
plt.xlabel('size of list'); plt.ylabel('Memory used')
```

sys.getsizeof or __sizeof__() only takes account of the list itself, not items it contains.

Difference between __sizeof__() and getsizeof()

The **__sizeof__()** method and the **getsizeof()** method both are used to get the size of the objects used in the program. The **getsizeof()** method returns an additional overhead for garbage collection along with each element size of the list. The **__sizeof__()** method returns the actual size of the object without any overhead. In this article, we will see how we can differentiate these operators in Python.

__sizeof__() operator	**getsizeof()**
The __sizeof__() operator returns the size of the object without any extra overhead for garbage collection.	The getsizeof() operator returns the size of the object with extra overhead for garbage collection.
It returns 40 bytes for an empty list and 8 bytes for each additional list element.	It returns 64 bytes for an empty list and 8 bytes for each additional list element.
It is not system specific and returns the same value on any system.	It is a system specific method i.e value returned by this method depends on the type of system it is executed on.
Importing the sys module is not required for __sizeof__() function to execute.	The sys module is required before using getsizeof() method.

More List methods and functions

- **len(list)** # the length of a list
- **sum(numbers)** # the sum of a list of numbers
- **any([1,0,1,0,1])** # are any of these values true?
- **all([1,0,1,0,1])** # are all of these values true?
- **list.append(5)** #add an element to the end:
- **list.count(5)** #count how many times a value appears:
- **list.extend([56, 2, 12])** # append several values at once
- **list.index(3)** # find the index of a value, if the value appears more than once, we will get the index of the first one; if the value is not in the list, we will get a ValueError!
- **list.insert(0, 45)** # insert 45 at the beginning of the list
- **my_number = numbers.pop(0)** # remove an element by its index and assign it to a variable
- **numbers.remove(12)** # remove an element by its value, if the value appears more than once, only the first one will be removed

Tuple

- Python has another sequence type which is called tuple. Tuples are similar to lists in many ways, but they are immutable. We define a tuple literal by putting a comma-separated list of values inside round brackets ().
- We can use tuples in much the same way as we use lists, except that **we can't modify them**. Tuple is used to create a sequence of values that we don't want to modify.

```
WEEKDAYS = ('Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday')
```

Set

- Python uses ***set*** to collect **unique** elements. If we add multiple copies of the same element to a set, the duplicates will be eliminated.
- To define a set, we need to put a comma-separated list of values inside curly brackets {}, and we can perform various operations on the sets.
- It is important to note that unlike lists and tuples sets are **not ordered**. When we print a set, the order of the elements will be random. If we want to process the contents of a set in a particular order, we will first need to convert it to a list or tuple and sort it.

```
even_numbers = {2, 4, 6, 8, 10}
big_numbers = {6, 7, 8, 9, 10}

# subtraction: big numbers which are not even
print(big_numbers - even_numbers)

# union: numbers which are big or even
print(big_numbers | even_numbers)

# intersection: numbers which are big and even
print(big_numbers & even_numbers)

# numbers which are big or even but not both
print(big_numbers ^ even_numbers)
```

range

- In Python, range is another kind of **immutable** sequence type. It is very specialized – we use it to create ranges of integers. It is mainly used in for loop.
- Ranges are generators. The numbers in a range are generated one at a time as they are needed, and not all at once.

```
# print the integers from 0 to 9
print(list(range(10)))

# print the integers from 1 to 10
print(list(range(1, 11)))

# print the odd integers from 1 to 10
print(list(range(1, 11, 2)))
```

```
#use range() in the for loop
for i in range(10):
    print("i=",i**2)
```

dictionary

- In Python, the dictionary type is called **dict**. We can use a dictionary to store key-value pairs.
- To define a dictionary literal, we put a comma-separated list of **key-value pairs** between curly brackets {}. We can use a colon to separate each key from its value.
- We can access values in the dictionary in the same way as list or tuple elements, but we use keys instead of indices.
- The keys of a dictionary don't have to be strings – they can be any immutable type. **Keys must be unique**, but when we store a value in a dictionary, the key doesn't have to exist – it will be created automatically.
- Like sets, dictionaries are not ordered – if we print a dictionary, the order will be random.

Example of dictionary

```
marbles = {"red": 34, "green": 30, "brown": 31, "yellow": 29 }

# Get a value by its key, or None if it doesn't exist
marbles["red"]
marbles.get("orange")
# We can specify a different default
marbles.get("orange", 0)

# Add several items to the dictionary at once
marbles.update({"orange": 34, "blue": 23, "purple": 36})

# All the keys in the dictionary
marbles.keys()
# All the values in the dictionary
marbles.values()
# All the items in the dictionary
marbles.items()
```

Containers: Arrays

- **Array**, is also an ordered set of values, but there are some important differences between lists and arrays. 1) The number of elements in an array is fixed; 2) The element of an array must all be of the same type.

- Advantages of **arrays**: 1) Arrays can be two-dimensional, like matrices in algebra; 2) Arrays behave roughly like vectors or matrices: you can do arithmetic with them; 3) Arrays work faster than lists.

- *a = numpy.zeros([m,n], complex),* create a 2D complex array with *m* rows and *n* columns

- *a = numpy.array(r, float)*, convert a list *r* to be an array *a*

- *a[m,n,k]* or *a[m][n][k]* the element in a 3D array

Some examples

```
>>> A=[[1, 2, 3, 'r', 'tet'],[5,4]]
>>> A
[[1, 2, 3, 'r', 'tet'], [5, 4]]
>>> A[0]
[1, 2, 3, 'r', 'tet']
>>> A[1]
[5, 4]
>>> A[0][2]
3
```

```
>>> C=np.array(A[1])
>>> C
array([5, 4])
>>> type(C[1])
<class 'numpy.int64'>
```

```
>>> A
[[1, 2, 3, 'r', 'tet'], [5, 4]]
>>> B=np.array(A)
>>> B
array([list([1, 2, 3, 'r', 'tet']), list([5, 4])], dtype=object)
>>> type(B[1])
<class 'list'>
>>> type(B[1][1])
<class 'int'>
>>> B[1][1]
4
>>> B[0][1]
2
```

Some examples

```
>>> import numpy as np
>>> A=np.zeros([3,3],int)
>>> A
array([[0, 0, 0],
       [0, 0, 0],
       [0, 0, 0]])
>>> A[0,0]=1.5
>>> A
array([[1, 0, 0],
       [0, 0, 0],
       [0, 0, 0]])
```

Quiz

- We prepare a 3D list in the following code. Please find out how many 0,1 and -1 in the 3D list.

```
from random import randint, seed
n=20;m=30;p=40;
seed(10)
A =[[[randint(-1,1) for x in range(n)] for y in range(m)] for z in range(p)]

### write your codes bellow to get the answer
```